Voice Arena — Optional Speaker Diarization Experiment

Goal: Evaluate two named diarization systems on **2–3 short public recordings** with human-annotated RTTM ground truth. Generate DER, JER, speaker-count error, prediction RTTMs, and reproducible result files.

Models: `pyannote/speaker-diarization-3.1` and `pyannote/speaker-diarization-community-1` (run only models for which your Hugging Face account has accepted the required terms).

Scoring: no collar, overlap included, no oracle speaker count; evaluate on the full clip unless a UEM file defines the scored region.

Important: This notebook contains **no experimental results**. Run it on actual licensed audio and reference labels before reporting scores. Google Colab's GPU runtime is recommended; models may take several minutes to download. Avoid uploading private calls or sensitive recordings.

1. Install dependencies

Run this cell once per Colab runtime. Restart the runtime if Colab asks you to.

```
!pip -q install -U "pyannote.audio>=3.3" "pyannote.metrics>=3.2" "soundfile>
```

```
ERROR: pip's dependency resolver does not currently take into account all the
google-colab 1.0.0 requires pandas==2.2.3, but you have pandas 3.1.1 which is
dask-cudf-cu13 26.6.0 requires pandas<2.4.0,>=2.0, but you have pandas 3.1.1
numba 0.64.0 requires numpy<2.5,>=1.22, but you have numpy 2.5.4 which is inc
google-adk 2.7.1 requires opentelemetry-api<=1.42.1,>=1.39, but you have open
google-adk 2.7.1 requires opentelemetry-sdk<=1.42.1,>=1.39, but you have open
datasets 4.8.5 requires huggingface-hub<2.0,>=0.25.0, but you have huggingfac
cudf-cu13 26.6.0 requires pandas<2.4.0,>=2.0, but you have pandas 3.1.1 which
```

```
%pip install -q --force-reinstall \
    "pandas==2.2.3" \
    "numpy==2.4.2" \
    "opentelemetry-api==1.42.1" \
    "opentelemetry-sdk==1.42.1" \
    "huggingface-hub>=0.25.0,<2.0"
```

```
ERROR: pip's dependency resolver does not currently take into account all the
pytorch-metric-learning 3.0.2 requires numpy>=2.5.3, but you have numpy 2.4.2
opentelemetry-exporter-otlp-proto-grpc 1.45.2 requires opentelemetry-sdk~=1.4
opentelemetry-exporter-otlp-proto-http 1.45.2 requires opentelemetry-sdk~=1.4
```

```
opentelemetry-exporter-otlp-common 0.66b2 requires opentelemetry-sdk~=1.45.2,
betterproto 2.0.0b7 requires python-dateutil<3.0,>=2.8, but you have python-d
gcsfs 2025.12.0 requires fsspec==2025.12.0, but you have fsspec 2026.9.0 whic
bigframes 2.51.0 requires python-dateutil<3,>=2.8.2, but you have python-date
ibis-framework 9.5.0 requires python-dateutil<3,>=2.8.2, but you have python-
google-cloud-bigquery 3.47.0 requires python-dateutil<3.0.0,>=2.8.2, but you
datasets 4.8.5 requires fsspec[http]<=2026.2.0,>=2023.1.0, but you have fsspe
holidays 0.108 requires python-dateutil<3,>=2.9.0.post0, but you have python-
```

```
%pip install \
    "python-dateutil==2.9.0.post0" \
    "fsspec==2025.12.0" \
    "gcsfs==2025.12.0"
```

```
Collecting python-dateutil==2.9.0.post0
  Downloading python_dateutil-2.9.0.post0-py2.py3-none-any.whl.metadata (8.4
Collecting fsspec==2025.12.0
  Downloading fsspec-2025.12.0-py3-none-any.whl.metadata (10 kB)
Requirement already satisfied: gcsfs==2025.12.0 in /usr/local/lib/python3.13/
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.13/dist-pac
Requirement already satisfied: aiohttp!=4.0.0a0,!=4.0.0a1 in /usr/local/lib/p
Requirement already satisfied: decorator>4.1.2 in /usr/local/lib/python3.13/d
Requirement already satisfied: google-auth>=1.2 in /usr/local/lib/python3.13/
Requirement already satisfied: google-auth-oauthlib in /usr/local/lib/python3
Requirement already satisfied: google-cloud-storage in /usr/local/lib/python3
Requirement already satisfied: google-cloud-storage-control in /usr/local/lib
Requirement already satisfied: requests in /usr/local/lib/python3.13/dist-pac
Requirement already satisfied: aiohappyeyeballs>=2.5.0 in /usr/local/lib/pyth
Requirement already satisfied: aiosignal>=1.4.0 in /usr/local/lib/python3.13/
Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.13/dis
Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.13
Requirement already satisfied: multidict<8.0,>=4.5 in /usr/local/lib/python3.
Requirement already satisfied: propcache>=0.2.0 in /usr/local/lib/python3.13/
Requirement already satisfied: yarl<2.0,>=1.25.1 in /usr/local/lib/python3.13
Requirement already satisfied: idna>=2.0 in /usr/local/lib/python3.13/dist-pa
Requirement already satisfied: pyasn1-modules>=0.2.1 in /usr/local/lib/python
Requirement already satisfied: cryptography>=38.0.3 in /usr/local/lib/python3
Requirement already satisfied: rsa<5,>=3.1.4 in /usr/local/lib/python3.13/dis
Requirement already satisfied: pyasn1>=0.1.3 in /usr/local/lib/python3.13/dis
Requirement already satisfied: cffi>=2.0.0 in /usr/local/lib/python3.13/dist-
Requirement already satisfied: pycparser in /usr/local/lib/python3.13/dist-pa
Requirement already satisfied: requests-oauthlib>=0.7.0 in /usr/local/lib/pyt
Requirement already satisfied: oauthlib>=3.0.0 in /usr/local/lib/python3.13/d
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/pyt
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.1
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.1
Requirement already satisfied: google-api-core<3.0.0,>=2.27.0 in /usr/local/l
Requirement already satisfied: google-cloud-core<3.0.0,>=2.4.2 in /usr/local/
Requirement already satisfied: google-resumable-media<3.0.0,>=2.7.2 in /usr/l
Requirement already satisfied: google-crc32c<2.0.0,>=1.6.0 in /usr/local/lib/
Requirement already satisfied: googleapis-common-protos<2.0.0,>=1.63.2 in /us
Requirement already satisfied: protobuf<8.0.0,>=4.25.8 in /usr/local/lib/pyth
Requirement already satisfied: proto-plus<2.0.0,>=1.22.3 in /usr/local/lib/py
Requirement already satisfied: grpcio<2.0.0,>=1.59.0 in /usr/local/lib/python
Requirement already satisfied: grpc-google-iam-v1<1.0.0,>=0.14.2 in /usr/loca
Requirement already satisfied: grpcio-status<2.0.0,>=1.33.2 in /usr/local/lib
Requirement already satisfied: typing-extensions~=4.12 in /usr/local/lib/pyth
Downloading python_dateutil-2.9.0.post0-py2.py3-none-any.whl (229 kB)
```

```
Downloading fsspec-2025.12.0-py3-none-any.whl (201 kB)
Installing collected packages: python-dateutil, fsspec
  Attempting uninstall: python-dateutil
    Found existing installation: python-dateutil 4.0.0
    Uninstalling python-dateutil-4.0.0:
      Successfully uninstalled python-dateutil-4.0.0
  Attempting uninstall: fsspec
    Found existing installation: fsspec 2026.9.0
    Uninstalling fsspec-2026.9.0:
      Successfully uninstalled fsspec-2026.9.0
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2/2 [fsspec]
Successfully installed fsspec-2025.12.0 python-dateutil-2.9.0.post0
```

2. Authenticate with Hugging Face

Before running, visit the model pages and accept their access conditions if required:

- https://huggingface.co/pyannote/speaker-diarization-3.1
- https://huggingface.co/pyannote/speaker-diarization-community-1

Create a Hugging Face read token. Colab's Secrets panel can store it as `HF_TOKEN`, or use the prompt below. Never paste the token into a shared notebook.

```
import os, getpass
try:
    from google.colab import userdata
    token = userdata.get("HF_TOKEN")
except Exception:
    token = None
if not token:
    token = getpass.getpass("Hugging Face read token: ")
os.environ["HF_TOKEN"] = token
print("Token provided:", bool(token))
```

```
Hugging Face read token: ··········
Token provided: True
```

3. Upload a ZIP containing audio and ground truth

Recommended directory structure inside your ZIP:

```
pilot_data/
  clip_01.wav
  clip_01.rttm
  clip_02.wav
  clip_02.rttm
  clip_03.wav
  clip_03.rttm
```

Use matching audio and official RTTM annotations from **AMI** or **VoxConverse**:

- AMI: https://groups.inf.ed.ac.uk/ami/corpus/
- VoxConverse: https://github.com/joonson/voxconverse

Do not treat diarization output as ground truth. Make sure RTTM timestamps align with the exact audio excerpts. If you trim an audio file, adjust RTTM timestamps and evaluation regions accordingly.

You can upload files directly below, or change `DATA_DIR` to a folder in mounted Google Drive.

```
from huggingface_hub import snapshot_download

dataset_path = snapshot_download(
    repo_id="ggfox00000/dia-AvaAvd-test",
    repo_type="dataset",
    allow_patterns=[
        "audio/test/*.wav",
        "rttm/test/*.rttm"
    ],
    local_dir="/content/ava_dataset"
)

print("Dataset downloaded!")
```

```
Download complete:         439MB, 55.1MB/s
Reconstruction complete: 100%        511MB /   511MB, 56.0MB/s
Fetching 108 files: 100%                                  108/108 [00:14<00:00, 21.41it/s]
Dataset downloaded!
```

```
from pathlib import Path
import shutil
import zipfile

source = Path("/content/ava_dataset")
output = Path("/content/pilot_data")
output.mkdir(exist_ok=True)

wav_files = sorted((source / "audio/test").glob("*.wav"))

count = 0

for wav in wav_files:
    rttm = source / "rttm/test" / f"{wav.stem}.rttm"

    if not rttm.exists():
        continue

    shutil.copy2(wav, output / wav.name)
```

```
        shutil.copy2(rttm, output / rttm.name)

        count += 1
        if count == 5:
            break

    if count == 0:
        raise RuntimeError("No matching WAV/RTTM pairs found.")

    zip_path = "/content/pilot_data.zip"

    with zipfile.ZipFile(zip_path, "w", zipfile.ZIP_DEFLATED) as z:
        for file in output.iterdir():
            z.write(file, arcname=f"pilot_data/{file.name}")

    print(f"Created {zip_path} with {count} matching pairs")
```

```
Created /content/pilot_data.zip with 5 matching pairs
```

```
from google.colab import files

files.download("/content/pilot_data.zip")
```

```
from google.colab import files
from pathlib import Path
import zipfile, shutil
uploaded = files.upload()  # select a ZIP of your recordings and matching RT
zip_names = [name for name in uploaded if name.lower().endswith(".zip")]
assert zip_names, "Please upload a .zip containing WAV files and reference R
DATA_DIR = Path("/content/voice_arena_data")
DATA_DIR.mkdir(exist_ok=True)
with zipfile.ZipFile(zip_names[0]) as z:
    # Reject paths that attempt to escape the extraction directory.
    for member in z.infolist():
        dest = (DATA_DIR / member.filename).resolve()
        if not dest.is_relative_to(DATA_DIR.resolve()):
            raise ValueError(f"Unsafe archive member: {member.filename}")
    z.extractall(DATA_DIR)
print("Extracted", len(list(DATA_DIR.rglob("*"))), "entries to", DATA_DIR)
```

```
Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving pilot_data.zip to pilot_data (1).zip
Extracted 11 entries to /content/voice_arena_data
```

4. Discover and validate the recording manifest

This expects each WAV and RTTM to have the same basename. Use only 2–3 short clips for the pilot.

```
import pandas as pd, soundfile as sf, hashlib
wavs = sorted(DATA_DIR.rglob("*.wav"))
rows = []
for wav in wavs:
    rttm = wav.with_suffix(".rttm")
    if not rttm.exists():
        print("Skipping (no matching RTTM):", wav)
        continue
    info = sf.info(str(wav))
    rows.append({"id":wav.stem, "audio":str(wav), "reference_rttm":str(rttm)
                 "duration_s":round(info.duration,2), "sample_rate":info.sam
manifest = pd.DataFrame(rows)
assert 2 <= len(manifest) <= 5, f"Expected 2–5 WAV/RTTM pairs; found {len(ma
assert manifest.id.is_unique, "Clip IDs must be unique"
display(manifest)
manifest.to_csv("/content/manifest.csv",index=False)
```

	id	audio	
0	1j20qq1JyX4_c_01	/content/voice_arena_data/pilot_data/1j20qq1Jy...	/content/vo
1	1j20qq1JyX4_c_02	/content/voice_arena_data/pilot_data/1j20qq1Jy...	/content/vo
2	1j20qq1JyX4_c_03	/content/voice_arena_data/pilot_data/1j20qq1Jy...	/content/vo
3	2qQs3Y9OJX0_c_01	/content/voice_arena_data/pilot_data/2qQs3Y9OJ...	/content/voice
4	2qQs3Y9OJX0_c_02	/content/voice_arena_data/pilot_data/2qQs3Y9OJ...	/content/voice

5. Parse RTTM and define the exact scoring region

Speaker IDs are arbitrary labels; the scorer optimally maps predicted IDs to reference IDs. The default scored region is the whole recording.

```
from pyannote.core import Annotation, Segment, Timeline
from pyannote.metrics.diarization import DiarizationErrorRate, JaccardErrorR

def read_rttm(path, clip_id):
    ann = Annotation(uri=clip_id)
    with open(path, encoding="utf-8") as f:
        for line in f:
            if not line.strip() or line.startswith("#"):
                continue
            x = line.split()
            if x[0] != "SPEAKER":
                continue
            if x[1] != clip_id:
                # Some public datasets use a different recording ID; this is
                # when the RTTM was verified to match this exact WAV.
                pass
            start, dur, speaker = float(x[3]), float(x[4]), x[7]
            if dur > 0:
                ann[Segment(start, start + dur), speaker] = speaker
```

```
    return ann

def write_rttm(ann, path, clip_id):
    with open(path, "w", encoding="utf-8") as f:
        for segment, _, label in ann.itertracks(yield_label=True):
            f.write(f"SPEAKER {clip_id} 1 {segment.start:.3f} {segment.durat

def as_annotation(output):
    if isinstance(output, Annotation):
        return output
    if hasattr(output, "speaker_diarization"):
        return output.speaker_diarization
    raise TypeError(f"Unknown pipeline output type: {type(output)}")

def sha256(path):
    h=hashlib.sha256()
    with open(path,"rb") as f:
        for chunk in iter(lambda:f.read(1024*1024),b""):
            h.update(chunk)
    return h.hexdigest()

for r in manifest.itertuples():
    ann=read_rttm(r.reference_rttm,r.id)
    assert len(ann)>0, f"No speaker annotations in {r.reference_rttm}"
    assert max(s.end for s in ann.get_timeline()) <= r.duration_s + 0.1, f"R
print("Reference RTTMs parsed successfully.")
```

```
Reference RTTMs parsed successfully.
```

6. Load both diarization pipelines

Model APIs can change. This cell tries the modern `token=` signature first and falls back to `use_auth_token=` for older installations. If a model is gated, verify your Hugging Face account has accepted its terms. **No known speaker count is passed to either system.**

```
import torch, time, platform, importlib.metadata
from pyannote.audio import Pipeline

MODELS = ["pyannote/speaker-diarization-3.1",
          "pyannote/speaker-diarization-community-1"]
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Device:", DEVICE)
print("pyannote.audio:", importlib.metadata.version("pyannote.audio"))
print("pyannote.metrics:", importlib.metadata.version("pyannote.metrics"))

def load_pipeline(name):
    try:
        pipeline=Pipeline.from_pretrained(name, token=os.environ["HF_TOKEN"]
    except TypeError:
        pipeline=Pipeline.from_pretrained(name, use_auth_token=os.environ["H
```

```
    if pipeline is None:
        raise RuntimeError(f"Unable to load {name}; check access and token")
    return pipeline.to(DEVICE)

pipelines={}
for name in MODELS:
    print("Loading:",name)
    pipelines[name]=load_pipeline(name)
print("Both pipelines loaded.")
```

```
Device: cuda
pyannote.audio: 4.0.7
pyannote.metrics: 4.2.0
Loading: pyannote/speaker-diarization-3.1
config.yaml: 100% 469/469 [00:00<00:00, 44.7kB/s]
pytorch_model.bin: reconstructing file: 100% 5.91MB / 5.91MB, 584kB/s
pytorch_model.bin: downloading bytes: 5.58MB, 552kB/s
plda/xvec_transform.npz: reconstructing file: 100% 134kB / 134kB, 13.3kB/s
plda/xvec_transform.npz: downloading bytes: 127kB, 12.5kB/s
plda/plda.npz: reconstructing file: 100% 134kB / 134kB, 13.2kB/s
plda/plda.npz: downloading bytes: 134kB, 13.2kB/s
pytorch_model.bin: reconstructing file: 100% 26.6MB / 26.6MB, 2.63MB/s
pytorch_model.bin: downloading bytes: 25.2MB, 2.48MB/s
Loading: pyannote/speaker-diarization-community-1
config.yaml: 100% 444/444 [00:00<00:00, 26.0kB/s]
segmentation/pytorch_model.bin: reconstructing file: 100% 5.91MB / 5.91MB, 581kB/s
segmentation/pytorch_model.bin: downloading bytes: 5.59MB, 549kB/s
embedding/pytorch_model.bin: reconstructing file: 100% 26.6MB / 26.6MB, 2.61MB/s
embedding/pytorch_model.bin: downloading bytes: 25.2MB, 2.46MB/s
Both pipelines loaded.
```

7. Run inference and calculate DER/JER

DER is the total false-alarm, missed-speech, and speaker-confusion time divided by reference speaker-time. **JER** summarizes per-speaker overlap disagreement. We report each clip separately; the pilot is too small to establish a statistically reliable model winner.

`collar=0.0`, `skip_overlap=False`, and the full recording timeline are explicit. This notebook does not use oracle speaker counts or automatic threshold tuning.

```
from pathlib import Path
import json
OUT=Path("/content/voice_arena_results")
OUT.mkdir(exist_ok=True)
results=[]
for name, pipeline in pipelines.items():
    model_slug=name.replace("/","__")
    for r in manifest.itertuples():
        print(f"Running {name} on {r.id}...")
        t0=time.perf_counter()
        output=pipeline(r.audio)
        runtime_s=time.perf_counter()-t0
        hypothesis=as_annotation(output)
        reference=read_rttm(r.reference_rttm,r.id)
        uem=Timeline([Segment(0.0,float(r.duration_s))], uri=r.id)
        der_metric=DiarizationErrorRate(collar=0.0,skip_overlap=False)
        jer_metric=JaccardErrorRate(collar=0.0,skip_overlap=False)
        der=float(der_metric(reference,hypothesis,uem=uem))
        jer=float(jer_metric(reference,hypothesis,uem=uem))
        prediction_file=OUT/f"{r.id}__{model_slug}.rttm"
        write_rttm(hypothesis,prediction_file,r.id)
        results.append({
            "clip_id":r.id, "model":name, "duration_s":r.duration_s,
            "der_pct":round(der*100,3), "jer_pct":round(jer*100,3),
            "reference_speakers":len(reference.labels()),
            "predicted_speakers":len(hypothesis.labels()),
            "speaker_count_abs_error":abs(len(reference.labels())-len(hypoth
            "inference_seconds":round(runtime_s,2),
            "audio_sha256":sha256(r.audio),
            "reference_sha256":sha256(r.reference_rttm),
            "prediction_rttm":str(prediction_file)
        })
        print(f"  DER={der*100:.2f}% | JER={jer*100:.2f}%")
df=pd.DataFrame(results)
display(df[["clip_id","model","der_pct","jer_pct","reference_speakers",
            "predicted_speakers","speaker_count_abs_error","inference_second
df.to_csv(OUT/"results.csv",index=False)
with open(OUT/"results.json","w") as f:
    json.dump({"models":MODELS,"device":str(DEVICE),"pyannote_audio_version"
               "pyannote_metrics_version":importlib.metadata.version("pyanno
               "scoring":{"collar_seconds":0.0,"skip_overlap":False,"speaker
               "results":results},f,indent=2)
```

```
Running pyannote/speaker-diarization-3.1 on 1j20qq1JyX4_c_01...
/usr/local/lib/python3.13/dist-packages/pyannote/audio/utils/reproducibility.
It can be re-enabled by calling
   >>> import torch
   >>> torch.backends.cuda.matmul.allow_tf32 = True
   >>> torch.backends.cudnn.allow_tf32 = True
See https://github.com/pyannote/pyannote-audio/issues/1370 for more details.

  warnings.warn(
/usr/local/lib/python3.13/dist-packages/pyannote/audio/models/blocks/pooling.
  std = sequences.std(dim=-1, correction=1)
  DER=63.16% | JER=89.54%
Running pyannote/speaker-diarization-3.1 on 1j20qq1JyX4_c_02...
  DER=73.15% | JER=83.27%
Running pyannote/speaker-diarization-3.1 on 1j20qq1JyX4_c_03...
  DER=51.83% | JER=73.99%
Running pyannote/speaker-diarization-3.1 on 2qQs3Y9OJX0_c_01...
  DER=74.89% | JER=86.43%
Running pyannote/speaker-diarization-3.1 on 2qQs3Y9OJX0_c_02...
  DER=80.20% | JER=93.36%
Running pyannote/speaker-diarization-community-1 on 1j20qq1JyX4_c_01...
/usr/local/lib/python3.13/dist-packages/pyannote/audio/models/blocks/pooling.
  std = sequences.std(dim=-1, correction=1)
  DER=56.12% | JER=81.40%
Running pyannote/speaker-diarization-community-1 on 1j20qq1JyX4_c_02...
  DER=56.13% | JER=71.75%
Running pyannote/speaker-diarization-community-1 on 1j20qq1JyX4_c_03...
  DER=51.59% | JER=72.86%
Running pyannote/speaker-diarization-community-1 on 2qQs3Y9OJX0_c_01...
  DER=63.65% | JER=70.37%
Running pyannote/speaker-diarization-community-1 on 2qQs3Y9OJX0_c_02...
  DER=53.42% | JER=61.91%
```

	clip_id	model	der_pct	jer_pct	reference_speakers
0	1j20qq1JyX4_c_01	pyannote/speaker-diarization-3.1	63.159	89.536	11
1	1j20qq1JyX4_c_02	pyannote/speaker-diarization-3.1	73.155	83.269	4
2	1j20qq1JyX4_c_03	pyannote/speaker-diarization-3.1	51.831	73.987	9
3	2qQs3Y9OJX0_c_01	pyannote/speaker-diarization-3.1	74.893	86.429	8
4	2qQs3Y9OJX0_c_02	pyannote/speaker-diarization-3.1	80.202	93.362	10
5	1j20qq1JyX4_c_01	pyannote/speaker-diarization-community-1	56.124	81.400	11
6	1j20qq1JyX4_c_02	pyannote/speaker-diarization-community-1	56.130	71.748	4

8. Compare errors visually

These are *actual computed results* only after the previous cell succeeds. A two- or three-recording pilot is for pipeline validation and qualitative error inspection, not publication-grade statistical significance.

```
import matplotlib.pyplot as plt
ax=df.pivot(index="clip_id",columns="model",values="der_pct").plot.bar(figsi
ax.set_ylabel("DER (%) — lower is better")
ax.set_title("Pilot diarization errors by recording")
plt.tight_layout()
plt.show()
display(df.groupby("model")[["der_pct","jer_pct","speaker_count_abs_error"]]
```



model	der_pct	jer_pct	speaker_count_abs_error
pyannote/speaker-diarization-3.1	68.65	85.32	5.6
pyannote/speaker-diarization-community-1	56.18	71.66	4.0

9. Download results and write your observations

Inspect the prediction RTTMs against the original audio, especially overlapping turns and short responses. In your assessment, record the exact model IDs, package versions, GPU/CPU environment, recording IDs, scoring settings, failures, and whether differences are consistent across clips.

Do not report a simple average of clip DER as corpus DER: corpus DER should be recomputed by summing the scoring error components across clips and dividing by

total reference speaker-time. This pilot intentionally presents per-clip DER and an *unweighted descriptive mean* only.

```
import shutil
archive=shutil.make_archive("/content/Voice_Arena_Pilot_Results","zip",OUT)
print("Created:",archive)
from google.colab import files
files.download(archive)
```

```
Created: /content/Voice_Arena_Pilot_Results.zip
```

Optional next improvements

1. Add a third documented system and compare outputs on identical recordings.
2. Supply official UEM scoring regions where available rather than scoring the full file.
3. Run the same RTTMs through `dscore` as a scorer cross-check.
4. Add a manual error-review table: missed overlap, speaker confusion, short-turn errors, and timestamp drift.
5. For a larger evaluation, compute **corpus-level** DER and paired recording-level bootstrap intervals. With 2–3 clips, confidence intervals are not credible evidence of superiority.

Research / documentation: AMI, VoxConverse, pyannote.metrics, dscore, DIHARD III.